Prompt compilation

Part 1 is the prompts that specified Sift, arranged by the feature they were for. A prompt is a parent when it opened that request. A later prompt is its child only when it changed or narrowed the same request. Words that only said go or start, and the lines Cursor inserts after a finished task, are not listed. They did not add a requirement.

Part 2 is the system text and the task text the product sends to a model. The hidden instructions that run the coding assistant are not part of Sift, and they are not in this document.

Contents

1. Foundation 3
2. Job draft, copilot drafting, and posting 5
3. CV intake, storage, and screening 7
4. Desk copilot 10
5. Interview room and the conducting agent 12
6. Face check and the interview room 14
7. Claim check 15
8. Consolidated assessment 16
9. Reports, comparison, and the interviews list 17
10. Ranking, invitations, and progress 18
11. Visual design 20
12. Diagrams, the SRS, and the innovation rationale 26
13. This compilation 28
System prompts 29

1. Foundation

The first instructions that set the platform shape and the rule that a phase is shown before it is built.

Parent

CURSOR INSTRUCTIONS: SIFT PLATFORM STEP-BY-STEP IMPLEMENTATION

You are acting as an expert Lead AI Software Engineer implementing the Sift platform according to the uploaded master document: `SIFT_WORKFLOW.pdf`.

System Architectural Principles & Guardrails
1. **Directory Structure**: Strictly maintain the dual `frontend/` (Next.js App Router) and `backend/` (FastAPI) project layout defined in Section 2 of `SIFT_WORKFLOW.pdf`.
2. **3-Tier Resilient LLM Fallback**: NEVER call Anthropic or Gemini APIs directly. All AI/LLM calls MUST route through `app.core.llm_fallback.FallbackLLMManager` (Claude 3.5 Sonnet -> Gemini 1.5 Pro -> Local Ollama llama3.2).
3. **Checklist Locking**: Once a Job Description is approved, its 10 criteria queries are frozen in PostgreSQL and cannot be altered without creating a new JD version.
4. **MCQ Score Isolation**: Total MCQ scores must ONLY be exposed to recruiter dashboards (`quiz_attempts` table) and NEVER returned in candidate-facing API payloads.
5. **Traceable Citations**: All assessment statements generated by LangGraph Workflow 4 must explicitly quote or reference transcript timestamps or CV text lines.
6. **Navigation Rule**: The `/login` page must strictly isolate and hide the global Navbar and persistent Copilot chatbot.

Execution & Testing Methodology
- We will implement the system **one feature at a time** strictly following the 7-Phase Roadmap in Section 4 of `SIFT_WORKFLOW.pdf`.
- For each feature request:
  1. Review the required file additions and dependencies.
  2. Generate production-grade, cleanly structured TypeScript/Python code.
  3. Include unit/integration tests or CLI verification scripts.
  4. Wait for explicit user test confirmation before proceeding to the next step.

CURRENT TASK STATUS: Awaiting

[The rest of this prompt continued in the same message.]

Child 1

there are errors on my terminal - fix them please

Child 2

so are we ready for phase 2?

Child 3

show me the plan for phase 2 befor executing it

Parent

@docs/architecture.md @.cursorrules

We are implementing Phase 1: FastAPI Skeleton, Health Endpoints, and OpenAPI configuration inside backend/.

Please implement the following files with complete, fully-typed code (no placeholders or TODOs):

1. `backend/app/api/v1/health.py`:
   - Define a GET `/health` endpoint returning `{"status": "healthy", "service": "sift-api", "version": "0.1.0"}`.
   - Fully typed with Pydantic schema for health response.

2. `backend/app/api/router.py`:
   - Initialize `APIRouter` with prefix `/api/v1`.
   - Include the health router.

3. `backend/app/main.py`:
   - Instantiate `FastAPI` app titled "Sift Candidate Screening API".
   - Configure CORS middleware.
   - Include main API router.
   - Configure OpenAPI doc routes at `/docs` and `/redoc`.

4. `backend/tests/test_health.py`:
   - Pytest unit test using `fastapi.testclient.TestClient`.
   - Asserts GET `/api/v1/health` returns HTTP 200 and expected JSON fields.

2. Job draft, copilot drafting, and posting

How a role is written, how the desk copilot may create it, and how an approved role is posted.

Parent

> also we would need to post the job somewhere keep that in mind - if you have some api for posting at a channel say liinkedin or any other please recommend and also make sure that you test everything yourself and understand the basis of the project since there are other phases to cover

Child 1

> https://github.com/postajob/api?utm_source=chatgpt.com could we use this for job posting - all we need is a demo post really

Child 2

> ok so that is what you're going to add with the rest of phase 2 - all good - let's begin phase 2

Parent

> implement it please

Child 1

> do you have any job drafts stored
> Stored jobs:
> - Front End Developer, version 1, draft
> - Backend Engineer, version 1, locked

Parent

> great but i need a major change - you know how we're posting on telegram using api - i want to implement some MCP feature there - meaning in want to post to an MCP server - so we use some sort of tool - please acknowledge if you're understood what i'm asking for and if not then please clarify before implementing it

Child 1

> can you elaborate 2

Child 2

> are you talking about building the server as well?

Child 3

> how are we gonna host it? is it too complex? and will it categorize as genuine client-server behaviour? will it be using JSONRPC and all

Child 4

> so this will categorize as implementing MCP since we are invoking a tool from a server?

Child 5

> ok that is what we're gonna do then before moving to the next phase - please test it - i will also test it later by posting something

Parent

> ok the chatbot or geenrate draft features aren't working - let's use this model - "gemini-2.5-flash-lite"

3. CV intake, storage, and screening

Mail and upload become a scored checklist, with a local recheck that may only raise a skill.

Parent

> Phase 2 is in place: policy upload, LangGraph job creation for both the Jobs form and the persistent copilot, checklist lock on approve, public careers page, and Telegram posting to @siftjobs when a draft is approved. The copilot stays as it is. Phase 3 is CV email intake, OCR, scoring against the locked checklist, top 3, and Mailpit invites. Do not rebuild the chatbot. Show the Phase 3 plan before writing code.
>
> this is a new chat - i just migrated to this chat to make sure the context is fresh since we will be starting phase 3 - please go through the document i've attached to be certain what we're doing and tell me what is phase 3 before we proceed

Child 1

> i need you to only focus on the file i uploaded just now the SIFT_WORKFLOW.pdf
> and understand this file - the thing is that you neeed to implement some system via you look out for mails with subject lines that you will save in the job tables and if you receive any - you will fetch CVs from those mails and then do the langgraph workflow with the CVs - please understand and acknowledge

Child 2

> what do you need from me to implement the 'CV from mail' and 'sending mail to chosen candidates' features - i need it to be just rigtht - also can you actually go through how we're planning on doing this mail's bit and also let me be clear - we will be using all free tools

Child 3

> i wanna test the whole thing out - create a job and post it and atttach a mail there so candidates can mail there and then, as a candidate, mail my cv back and then the whole workflow working and then assessing the candidate and adding them in the candidates section (also adding that in the frontend - navbar, it's own page and all with their score and all) showing their 'profile' which i can click on and see which job they applied for and all that
>
> also they should be grouped by the job they applied for

Child 4

> ok so i wanna test out the mail with an actual mail - 'kazisadmansakib@gmail.com' such that this mail is attached with the job description (all of the job posts now will have this) and also the consequent subject line so that when someone mails here i can check the mails and if the subject lines match then i will extract the CVs for the consequent jobs and then start the assessment pipeline for the CV - please acknowledge and understand

Child 5

> but i want to test it out in the real way = meaning as a candidate mails his cv on this mail i want my system to pick it up and then go forward with the pipeline

Child 6

> i want the system to assess and then in the candidates page add all the candidates, their score, ranking and all - we will

> refine this in the future but this is what i wnat for now

Child 7

> ok so here's a plan i'll make a custom email for sift (help me with the specifics please) and we'll use that to send and receive the mails = and also implement IMAP on that - sounds good?

Child 8

> i had the username wrong - hopefully it's right now - the mail is fizoadverts@gmail.com - so continue and finish the work

Child 9

> i need a way to save the CV as well perhaps the drive - or no?

Child 10

> but we might need to see their cv later - right? specially during the interview process so saving it is wise

Child 11

> wait you do realize that the cvs are being sent to fizoadverts@gmail.com from potentially any mail right?

Child 12

> ok so you do realize that you need to check mails for all the subject headlines against all the job we've created right?

Parent

> i need it to be better and use a model to do the embedding - also becaue the outputs are not that good - how else can we improve the quality

Child 1

> the system performs better however not as good as i thought for example the job experience in the cv add up to 11+ years - but combining 3-4 jobs however the screening says that that is not the case - what do you think is the solution - i was thinking using the LLM here as well and alot of other things are wrong for example the cv highlights the experience with pgvector however the screening says that there isn't sufficicnt evidence

Child 2

> but say someone says 'expertise in FastAPI' however i wanted 'python experience' will we able to catch that

Child 3

> can we somehow give this task to an LLM - the local one for that

Child 4

> meaning - only use the LLM to recheck for unmet, or partially met ones (since the ones that we have met are there) -

> does that sound like a good idea

Parent

> ok now i need one more feature and this perhaps should've been implemented way before - i have to be able to upload CVs from the site and this way the tests can also be done more smoothly and faster, for example i don't have to email and then wait for extraction, this however will work in the same way after uploading (and this can only be done for a speicific job post and perhaps 20 at a time) they will be assessed and also mails send with invites - please acknowledge

4. Desk copilot

The recruiter’s chat on the desk: look, navigate, draft, and refuse the guarded actions.

Parent

> let's work on the copilot now please i believe that it does have a few agentic feature - can you list every feature it has

Child 1

> can we give it all agentic power - starting from starting a practice call to redirecting me to any div on any page always?

Child 2

> ok so can we turn our copilot into an actual copilot capable of doing it all

Child 3

> what can it not do

Child 4

> a few more changes about the copilot
> - there should be streaming - meaning when it thinks those dots jumping in rhythm
> - it should produce one word after another instead of printing the whole output at once
> - also when i click on copilot the cursor immediately moves to the typing box so i can start typing immediately
> - change the color and outlook of the copilot icon that's floating there(bottom-right) - please use your own imagination and fix it

Child 5

> clicking on the copilot sort of brings it to the left - you need to fix it snad the icon looks weird - very 2D but white boundary is also ugly - please glassify it and make it slightly more aesthetic

Parent

> i just asked the copilot to take me to another page and view some candidates and it literally acted like an agent and redirected me - how is that happening - ruotes? regex? LLMs?

Child 1

> so the llm is used to make the url to hit the endpoint

Child 2

> there is no use of mcp or tools right?

Parent

> there seems to be an issue with the chatbot - i think there is some context limit - because after some time there's a 422, if i remember right - and also when asked to list all the shafat thsi happens (the screenshot) - how do we fiix these i wnat it

> to be better at understanding the user's commands

Parent

> as i write in the input box of the copilot the previous text is dissppeared as they are pushed towards the left but i want it to be such that it goes to next line if a line is full - much like it's happening here on this text box

5. Interview room and the conducting agent

The live call, and the agent that speaks when a human interviewer is not in the room.

Parent

> ok the copilot isn't there honestly i need you to, for now, not worry as much about the UI and built me the copilot that has access to the CV (profile of the person that we're interviewing) and the live transript and can be asked to generate questions based on what has been asked already and also what the CV claims as the person's skills and also - we will implement that for today as well - a few other things the recruiter should only be able to end the call and that ends the whole thing - the candidate could leave but that wouldn't end the call - please acknowledge all this

Parent

> i believe we have skipped the q&a analysis from phase 4 - please acknowledge

Child 1

> so let's do that now - first please acknowledge what we must do to complete this feature

Child 2

> you do realise this is where we will also implement langgarph workflow 3 right

Child 3

> ok a few issues:
> - after i press end interview - the interview seems to keep loading and doesn't end until i go to the previous page, i want it to end and then take me to the Q&A analysis and so on, this i also want for the practice interviews
> - the interview page is really clunky the page always forcefully brings me back below even if i try scrolling up
> - i tried entering as a candidate from my phone, all is good except the mute button doesn't work
>
> some suggestions:
> - i want the video interview interface to be like that of meet, meaning a huge screeen that shows the candidate and a smaller box for me on the right bottom side (perhaps can be dragged) the mute, decline call and all the features should be icons and right there as well
> - on the right side of this window is a vertical interface (somewhat wide - not as much that the main video conference seems small by any means) that can display both the transcript and the chatbot (that makes the questions) - we can switch between the two
> - so all and all - really no scope to scroll up and down
>
> ==========
> now despite all this understanding - please just take into account the gyst of it and use your own knowledge of UI/UX to accomplish all this
>
> please acknowledge

Parent

> the transcript isn't working - i'm speaking but no text on the transcript - please check

Child 1

> why does it take some time before the transcript starts to work

Child 2

> can we fix this delay

Parent

> i have made some changes to the way the recruiter agent works - it primarily and only uses gemini api and nothing local - can yo ucheck if that is indeed the case

Child 1

> and what are we using now for TTS

Parent

> a slight but important change please
> -for the transcipt the text should appear just like messenger chats - left chat boxes for candidate and right ones mine - and left ones should be denoted as the candidates actual name and the color of the text of both perhaps should be different too - please acknowledge

Parent

> this is the agent that conducts the interview dude - the one that takes over the interview when the human interviewer isn't present you need to add more detailsl to the rationale - which tools it used (i forgot komoro or something and all of that) please highlight it's importance and profoundness of this tech as well keeping in mind the actual requiremet of the rationale pdf

6. Face check and the interview room

Who is in the room, how often a face is judged, and how the room should look.

Parent

> i believe all core features and all phases are complete please read this "F7 —CandidateIdentification & Real-Time Profile Overview
> Identify the interviewee via facial and/or voice recognition, retrieve their stored information, and present a concise profile overview within the interview interface.
> ● Matchthepersonjoining the session against the candidate records already in the system.
> ● Onaconfidentmatch,surface a compact profile panel to the interviewer — name, headline experience, key skills, pre-interview score and flagged areas to probe.
> ● Handlenon-identification and low-confidence matches explicitly; never silently assume an identity.
> ● Treatbiometric data with particular care — see Section 7.3 on candidate data privacy. Consent handling must be visible in the flow."
> this will be our new feature that we will be working on today - please acknowledge, we will get images from CV and then save them across candidate id/name/profile and then later match them during live interview to confirm if in fact this is the candidate and if not thne who, from our database, is this most likely - please acknowledge

Child 1

> which stacks/models will we be using for these

Child 2

> here are a few changes that i want:
> - take a look at the videoroom ui (screenshot attached) - it is too crowded and the video frames literally move downwards - the assessments (similarity and all) are occupying the upper part which is ugly perhaps add it along with teh transcript and questions and instead of constantly doing it maybe do it 5-10 times in regular intervals and then reach a conclusion and then perhaps jsut show me the conlusion that is all - please we need to fix the UI as well - this is too crowded - please acknowledge

Child 3

> also the hue designi present in all the other pages isn't present in the video room and also the asssessment page that comes after the video chat - we need to integrate that as well - though in the video room the hue UI is only slightly seen due to the enormity of the video frames - you get it though - please acknowledge

Child 4

> ok the video room ui - perfect
> however the transcript seems to be very very slow - i need you to propose a better idea for face id match too

Child 5

> so what do we do now

7. Claim check

CV claims compared with what was actually said.

Parent

> F8 —CVClaimVerification Through Interview Analysis
> ● Cross-reference claims made in the CV against what the candidate said during the interview.
> ● Flaginconsistencies, contradictions and claims that were asserted but never substantiated.
> ● Distinguish clearly between a genuine contradiction, an unverified claim and a claim that was simply never discussed.
> ● Presentflags as prompts for humanfollow-up, never as accusations or automated rejections.
>
> we will now work on this - please acknowledge

Child 1

> how will we achieve this

Child 2

> are we using LLMs here or what and which one

8. Consolidated assessment

The cited mix of CV, quiz, and interview.

Parent

> Sift is at F:\Final Project\sift. Phase 5 (dynamic MCQ quiz) is done. Phase 6 is architecture section 11: LangGraph Workflow 4, consolidated candidate assessment (BID F6).
>
> Already stored and ready to load:
> - CV screening snapshot and per-question scores (Phase 3)
> - mcq_attempts.score, 0–100, recruiter-only (Phase 5). Candidate payloads must never include the total.
> - Interview Q&A analysis and qa_pairs after the recruiter ends the call (Workflow 3). Candidates do not see it.
>
> Not built: consolidated assessment graph, tables, API, or recruiter comparison view. docs/architecture.md section 11 is the spec (load CV, MCQ, and Q&A, then strengths, gaps, overall suitability, a traceable narrative, and persistence). It does not fix the mix weights.
>
> Rules: Python 3.11+, Pydantic v2, async SQLAlchemy, every LLM call through FallbackLLMManager, pytest for new modules, no placeholders. The site runs on Docker; source is copied into the images, so API or UI changes need `docker compose up -d --build api frontend`. Alembic runs on API startup. Current head is b2e8c4a91d60. Do not commit unless asked. Personal notes live outside the repo, next to sift-phase-5-notes.md.
>
> please acknowledge

Child 1

> are you ready to start building phase 6

Child 2

> how do i test

9. Reports, comparison, and the interviews list

What the recruiter reads after the call, and how two people are placed side by side.

Parent

> we need to add 2 more features:
>
> - Report view and export (PDF/HTML)
> - Side-by-side candidate comparison view
>
> the first allows the interviewer to take a look at the candidate's (that they are interviewing) CV while the interview is going on - perhaps add that along with the trancript, questions options
>
> the second allows us to compare 2 candidates that have applied for the same role - we can do this with both the chatbot (saying it takes us to the comparison page) or we can compare by selecting 2 CVs after entering a role - we do not add a separate page on teh navbar but rather this is in the candidates page - upon entering there and then entering a role we can compare 2 (the compare optino is with the candidates/standing options) we can choose 2 candidates (however you feel is best - must be easy and aesthetic please) and then the comparison occurs (perhaps we need some LLM here - you think through this part)
>
> please acknowledge

Child 1

> i dont see the compare

Parent

> ok now a sifnificant change - i want teh progress page replaced with interviews - that shows all the interviews for each role - only ones that actually have interviews scheduled - please acknowledge

10. Ranking, invitations, and progress

The recruiter chooses who is invited, and long work has to show its progress.

Parent

> i have another query - if i upload 20 CVs together - say the first CV is assessed since it is the first one to be ranked it automatically goes to top in ranking - does that mean they get sent the mail anyways - becuase maybe they weren't in the top and they are just selected because they hap[pen to be the first one to be selected - do we have this bug or are they all first ranked and the mailed invitations after - pleaese check

Child 1

> how do we solve this perhaps notify me about the ranking and then i will go to that page and i will have a bunch of choices to select top 3,4,5 or custom select them - only notifying me about the ranking/new applicants - does that sound good - please suggest if you have any thing better

Child 2

> so sending invitations now requires me in the loop

Child 3

> please make the changes

Parent

> before we start testing the whole platform - we need to make a few changes - the first is that of the notification system - after a mail is ingested i feel, or even after i hit check inbox it'st taking too long to process and i'm sittting blind pretty much - we have to solve this uncertainty for users - any suggesttion?

Child 1

> so a progress bar there as well?

Parent

> i went to a job and uploaded a cv after that, while the cv was being processed and that showed on the progress bar - i tried to go to home then the screen went black for quite a while and also after asking teh chatbot to take me to home the screen remained black for a long time and then appeared - the cv was screened though - why is this happening and how to solve it

Child 1

> ok fix it please

Parent

> ok so we need to fix a few things after the final ranking there is no part where we see the final ranking i mean the ones that have been through it all - teh cv assessment, interview, assessment and all i will finally send them an approval mail -

> but i need to decide how many of the top get the mail - and also the number i select is my own wish - we need to add the part where we get to see final ranking for a job - i bilieve that isn't here - please acknowledge

Child 1

> please fix that

Child 2

> i wanna be able to send invitations to these toppers and i should be able to select how many i sent invitations to (only from the top) and also i want to send video interview invitations to as many people as i want from the top (right now it's 3,5 or something and fixed) - please make sure that these are fixed and both the video invitation mail and final approval letter mails should be improved they must be realistic and good looking

11. Visual design

The look of the desk, the sign-in page, the logo, and where login lands.

Parent

I am redesigning the UI of my existing recruitment platform. I have provided a visual reference image that represents the aesthetic I want to achieve.

Your task is to translate its visual language into an original, premium recruitment product — not to copy the reference website literally.

1. VISUAL DIRECTION

Study the reference image carefully.

Pay attention to:
- The way orange, amber, coral and complementary colors blend into one another.
- The atmospheric background and its sense of depth.
- The contrast between sharp foreground content and softer, defocused background elements.
- The use of light, gradients, composition, typography and whitespace.
- How visual emphasis guides attention without making the interface feel cluttered.
- The balance between expressive art direction and readable functional content.

I do NOT want an overwhelmingly black interface, a single-color background, generic SaaS styling, or a page covered in purple gradients and glassmorphism.

I want a cinematic, premium, warm, layered visual identity with sophisticated color blending and selective focus.

The reference image is inspiration for the overall design language, not a literal template. Adapt its principles to the needs of recruitment software.

2. UNDERSTAND THE EXISTING PRODUCT

Before editing any files, inspect the repository.

The platform has 11 routes covering:
- Recruiter home.
- Recruiter login.
- Company policies.
- Job creation, editing, approval and applicant management.
- Candidate discovery and candidate profiles.
- Practice interviews.
- Careers listings and job details.
- Public interview invitations.
- A LiveKit-powered interview room with recruiter and candidate experiences.

A shared application layout includes recruiter authentication gating, navigation and a

[The rest of this prompt continued in the same message.]

Child 1

> Proceed with the shared design foundation and recruiter homepage only.
>
> Implement the agreed design tokens, typography, background treatments, shared navigation and reusable UI primitives using the existing project architecture.
>
> For the homepage, create a cinematic recruitment experience with warm blended gradients, subtle atmospheric blur, layered lighting, strong typography and deliberate composition. Use realistic sample content only where necessary, keeping it separate from the real data layer.
>
> The result must feel like a cohesive premium product, not a generic dashboard or a direct copy of the reference image.
>
> Preserve all existing functionality. Do not redesign other pages in this step.
>
> After implementation, run the available checks and inspect the rendered page if browser tooling is available. Report the changed files, verification results, any mocked behavior and any remaining issues.
>
> Stop after the homepage is complete so I can review it before we continue.

Child 2

> here's the thing - the colors might be slightly more enhanced please take a look at the current image(first) and reference image (second) again and try gettnig things right

Child 3

> the color is too light - the previous color was better and maybe slightly brighter in terms of the contrast and also the colors aren't accurate use the second image as reference the background has a blend of many colors yet sort of in the background giving a 3d sense and the glass interfaces in the front "closer to me" gives it all an aesthtic sense - try and imitate it please

Child 4

> can you now use this style for all pages and the entire platform - please make sure the style is consistent across all pages

Parent

> i like the UI except i feel it's too dark - could we use some coloring to make it slightly brighter - not too much though however the cards do remain transparent - perhaps slightly more so such that the light in the background can shine through - the cards of course should be darker than the backgound - let's try it out please

Child 1

> you know this amber/orange thingy isnt that much of fun to the eyes i guess - any suggesttions to make this better

Child 2

> could we try and make the cards more glass panelish - refer to the image i've attached - the color reflections are of course of the light color from the background

Parent

> now let's fix the candidates page please - when i enter there there are cards, each one representing a job and as i enter them, inside are many more cards each, a candidate with their pic on them (if present and extracted from cv however just a default one if not present) and their name below - a solid aesthetic UI that is and make sure after entering a job i can either view the standing or the candidates (they are also ranked in their card #1 or #2)
> just an ordered way to see it all
> please acknowledge and improve on this idea

Child 1

> the image is too zoomed in maybe keep it just the way it was in the cv that is better - this look too creepy

Parent

> take a look at the policy page - let's make it a bit more premiujm - the choose file option looks like raw HTML and also let';s use square cards to represent the different docs that have been uploaded and clicking on them maybe shows a tiny summary (use local LLM here please)
>
> and another major change i want is that the theme changer (that is now at teh bottom of the page) should be on the top right left to the notification icon and let's keep it in the shape it is

Parent

> this page also has not context and is too messed up meaning if i entered this page for the first time - i'd have no idea - also it feels kinda redundant to have this and another page called "Careers" let's make the Job page such that there is the option of check inbox however there are different tabs to either see existing jobs - the job cards pop up, upon entering i have tabs there as well where i can either see the describtion and questions or maybe see applicants there as well - pleaes use your imageination
> another tab in jobs is to create a job manually
>
> i'm not very clear about the UI and flow myself - please use your own imagination and build it

Child 1

> do we really need careers page

Child 2

> ok drop it

Parent

> the copilot seems to be lost in the UI - can we light it up - add some glowing effect - very minimalistic but enough to know it's there

Child 1

> i don't like it - perhaps the previous what better but add a very faint glow that circles the border of the copilot capsule - please acknowledge

Parent

Add a subtle dot-grid texture to the app's global background.

Context: The app (Sift) has a dark gradient background with four themes (Rose, Moon, Wine, Moss) and a lighter glow in the top-right corner. Find where the global background/theme is defined (likely the root layout, body styles, or global CSS) before editing.

Requirements:
1. Add a fixed, full-viewport overlay using a pseudo-element on the app root (body::before, or the root layout wrapper if body is not suitable). It must:
- be position: fixed; inset: 0
- have pointer-events: none
- sit above the background gradient but below all content (use z-index: 0 and make sure the app content is position: relative with a higher z-index, or use z-index: -1 if the stacking context allows)
2. Use this CSS as the starting point:

```
:root { --dot-color: rgba(255, 235, 240, 0.16); }

.app-bg::before {
  content: "";
  position: fixed;
  inset: 0;
  pointer-events: none;
  background-image: radial-gradient(var(--dot-color) 1px, transparent 1.3px);
  background-size: 16px 16px;
  -webkit-mask-image: linear-gradient(200deg, #000 0%, transparent 75%);
          mask-image: linear-gradient(200deg, #000 0%, transparent 75%);
}
```

The mask makes the dots fade out from the top-right toward the bottom-left, so they are strongest near the glow.
3. Make the dot color a CSS variable (--dot-color) and set a tinted value per theme, so the dots match each palette instead of always being pinkish-white:
- Wine / Rose: warm pink-white, around rgba(255, 235, 240, 0.16)
- Moon: cool blue-white, around rgba(225, 235, 255, 0.14)
- Moss: pale green-white, around rgba(230, 255, 240, 0.14)
Hook into however the themes are currently switched (data-theme attribute, class, or

[The rest of this prompt continued in the same message.]

Parent

Redesign Sift Sign-In — Cinematic Glassmorphism Using the Attached Artwork

Update the **existing Sift sign-in/sign-up page** to create a premium, cinematic authentication experience. Use the attached Sift artwork as the visual foundation and implement the surrounding interface, glass panel, ambient background, and animations directly in the current codebase.

Modify the existing design in place. Do not rebuild the application or replace working authentication functionality.

First, inspect the project structure, current authentication page, styling system, existing theme selector, and authentication logic. Then implement the redesign using the project's established conventions.

1. Creative direction

Create an immersive, editorial-style sign-in experience for Sift, an AI recruitment platform.

The visual identity should feel warm, cinematic, sophisticated, and confident. Build around the artwork's deep wine shadows, burgundy, dusty rose, muted plum, and golden-hour amber.

The overall experience should feel like a beautiful scene existing behind a carefully crafted piece of dark glass.

Avoid generic SaaS login layouts, bright white cards, excessive gradients, neon accents, and overdone glassmorphism.

2. Use the attached Sift image correctly

Use the attached image as the actual artwork for the left side of the authentication panel.

Locate the image in the project assets if it has already been added. If not, integrate the supplied asset using the project's existing asset conventions.

The image already contains:
- The Sift logo.
- The headline, "You set the bar. Sift finds the fit."
- Supporting recruitment-focused copy.
- The closing line, "Intelligent hiring starts here."
- The warm, motion-blurred crowd and sunset lighting.

**Do not recreate or overl

[The rest of this prompt continued in the same message.]

Child 1

> it's all goood but the background doesn't have the design all the other pages have that dotted effect and all - please appliy the same design in the signin background as is in otehr pages

Child 2

> i like this however there is too much space on top reduce that please - put the theme bar on top of the sign in card if necessary
> - in the interviews page, the theme bar is too much right to the sceen such that it fall behind the notification icon - make it like any other pages
> - in the candidates page the the inteviews text on the top right falles behind the theme bar - please fix that, maybe move the interviews a bit below than wehre they are

Child 3

> the background is too dark and not in contrast with the glass panel - let's fix that and only for the sign in page make the background more brighter and also the panel is not taking up too much space let's reduce that a bit more - let's take more space on the sides and less so above and below - use your own style sense though

Child 4

> - i asked to keep less space on sides and more above and below
> - not this light dude - slightly more than before and it should still hae the hue effects and all

Parent

> can you use the sift logo from the signin page image and use that for all pages (replacing the default logo that's on all the top-right of all pages)

Parent

> upon loggin in i go to the job page instead of home page - can you take a look please

12. Diagrams, the SRS, and the innovation rationale

The documents asked for after the product existed.

Parent

> ok now i need to now make all the diagrams you can produce from this project - the langgraphs, architecture diagrams (both high and low level) and also use case diagrams and such - can you make them? and if so can you do them well? i need to work on my presentation slides and also the SRS and so i need the diagrams - please acknowledge

Child 1

> for now i need you to make the langgraphs and architecture diagrams - and outside the project directory of course - please acknowledge

Child 2

> teh high level and low level aren't what i wanted - could you pull up actual project demos and make them - and i want all in pdf format so i can just download them - i like the langgraphs though - please fix the architecture diagrams a

Child 3

> i need you to take this image as a reference please - can you please take this design patter in to account and then analyse my project and make a similar design for my project - please take your time and construct the image as pdf maybe

Parent

> if i were to give you a sample of an SRS and tell you to make me one for this project - would you be able to do that

Child 1

> now this is an actual SRS of a project and it is absolutely essential that you do not plagiarise or copy any data from here - but only use it as a reference and guideline to create the PERFECT SRS for Sift - please acknowledge after analysing the sample SRS

Child 2

> it's not nearly big and detailed enough i have attached the @docs/SRS_Sample_2.pdf please analyse this and then analyse this project and then write the SRS in details

Parent

> ok now for a really crucial task - you've perfectly written the SRS however i need it to follow a certain template - i've attached @docs/SRS Template.doc pleaes use this template and put all of of @docs/Sift_SRS.pdf according to the template - make sure you don't leave anything from the SRS - just rearrange according to the template - take your time and get it right

Child 1

> can't you do pdf instead and make it ordered and all

Parent

> can you design slides for me

Parent

> next i need you write this: "Task: Write 300–500 word innovation rationale for /docs
> Task Description: Author the formal technical documentation explaining the purpose, architecture, and value of Feature F11. Technical Approach: Write a structured markdown/PDF document stored in /docs/innovation_rationale.pdf detailing problem statement and design choices.Task: Write 300–500 word innovation rationale for /docs
> Task Description: Author the formal technical documentation explaining the purpose, architecture, and value of Feature F11. Technical Approach: Write a structured markdown/PDF document stored in /docs/innovation_rationale.pdf detailing problem statement and design choices." this of course is the recruiter agent

Child 1

> this is the agent that conducts the interview dude - the one that takes over the interview when the human interviewer isn't present you need to add more detailsl to the rationale - which tools it used (i forgot komoro or something and all of that) please highlight it's importance and profoundness of this tech as well keeping in mind the actual requiremet of the rationale pdf

13. This compilation

The request for this document, and the order it had to follow.

Parent

> i need a compilatio of all my prompts - the system prompts that we've used and the parent-child prompts i've used to accomplish all this - is it possible to compile it all i one pdf

Child 1

> perfect - can we now prepare the whole compilcation for prompts

Child 2

> for the prompts that i've given can you arrange them according to feature (for which hte prompts were given) and also parent child(if indeed they were parent child) and then later comes the ssystem prompts portion - pleaes acknowledge

System prompts

Each block is the instruction stored in the product. Where a turn has two parts, the system line tells the model its role and the task line carries the record for that turn. Ordinary completions try Claude, then Gemini, then the local model. The spoken interview line asks Gemini only.

Interview agent — system instruction for the live turn

Sent as the system text. Gemini is the only model asked for this turn.

```
_SYSTEM = (
    "You are conducting a live video interview. You can see and hear the candidate. "
    "You speak aloud, so say is one or two short sentences. "
    "Never say you are a text interface, a chatbot, or an AI that cannot see or hear. "
    "If they ask whether you can see or hear them, say yes in a few words. "
    "Do not describe their face, clothes, or room. "
    "Return only a JSON object with keys kind (string), say (string), answered (boolean), "
    "covered (list of checklist items), done (boolean), and quiz (boolean). "
    "kind is your judgement of what they just did, one of answer, partial, question, clarify, aside, decline. "
    "Judge the meaning, not the punctuation. Speech-to-text often drops question marks. "
    "answer: they addressed the open question with something relevant, even briefly or imperfectly. "
    "partial: only yes, no, or a fragment with nothing specific. "
    "question: they asked you something, about the role, team, process, or anything else, instead of answering. "
    "clarify: they did not understand, asked you to repeat or explain the question, or asked what you mean. "
    "aside: a greeting, a comment, or small talk that is not an answer. "
    "decline: they would rather not answer. "
    "The prompt ends with a word-based guess of the kind. It is often wrong, so decide yourself. "
    "The checklist is a private list of topics. Never read a topic aloud, never say 'the candidate', and never say 'demonstrate'.
"
    "Ask one specific situation for the open topic: a bug, a failure, or a tradeoff in this role. "
    "For backend work, ask how they would find and fix a concrete failure. Do not ask whether they have experience. "
    "Sound calm. Do not start with 'So you', 'Okay', 'Right', or 'Got it'. "
    "Name one concrete detail they just said, in your own words. Do not repeat their sentence. "
    "Do not repeat names, companies, or other proper nouns from their last sentence. "
    "If you mention their background, use the CV's wording, and do that only when it helps this question. "
    "If their answer conflicts with the CV, mention that difference inside the next situation. "
    "If the answer also covers a later checklist item, include that exact item in covered. "
    "When the kind is answer, they have said enough. Set answered true. "
    "A related answer counts. Do not wait for particular words or a complete design. "
    "Then ask one specific situation about the next uncovered topic, and put the open item in covered. "
    "Do not ask the same topic again. "
    "For every other kind, set answered false, leave covered empty, and do not start a new checklist item. "
    "question: answer it naturally in one or two sentences, as a person would. Do not repeat their question back. "
    "You know nothing about the company, team, tools, salary, or process beyond this prompt. "
    "For those, say the recruiter will follow up on that. Never invent a fact. "
    "Then return to the open topic with a short bridge, such as 'Coming back to the outage', and ask it in your own words. "
    "clarify: explain the open question more simply, with a short concrete example of what you mean, "
    "and ask it again in different words. "
    "aside: respond briefly and warmly, then ask the open question in your own words. "
    "partial: ask one short follow-up that invites a concrete example. "
    "decline: say that is fine. "
    "Ask exactly one question per reply. "
    "If they have not spoken yet, set answered false and ask the first checklist item in your own words. "
```

```
    "Do not invent requirements. covered may only contain exact checklist items. "
    "When the checklist is covered, or the candidate has answered enough, set done and quiz to true "
    "and ask whether they have a question before the skill quiz."
)
```

Interview agent — system instruction after the questions are finished

Sent as the system text for the closing exchange.

```
_CLOSE_SYSTEM = (
    "The interview questions are finished. You are still on the video call and can see and hear them. "
    "Answer the candidate in one short spoken sentence. "
    "Never say you are a text interface, a chatbot, or an AI that cannot see or hear. "
    "If you cannot know, say so. Do not ask a new question. "
    "Return only a JSON object with the key say."
)
```

Interview agent — task text for one turn

The task is filled with the company, role, CV profile, checklist, and what was just heard.

```
return (
        f"Company: {company}\nRole: {job_title}\n"
        f"CV profile:\n{profile_text}\n\n"
        f"Checklist:\n{checklist}\n\n"
        f"Already asked:\n{asked}\n\n"
        f"Already covered:\n{covered}\n\n"
        f"Open checklist question:\n{open_item}\n\n"
        f"Transcript:\n{history}\n\n"
        f"Candidate just said:\n{heard.strip()}"
    )
```

Interview agent — opening line

Spoken before the first checklist question. It is fixed text, not a model reply.

```
return (
        f"Hello {name}. I am the AI interviewer for the {role} role at {company_name}. "
        "I will ask a few questions, and then there is a short skill quiz. Whenever you are ready."
    )
```

Interview agent — hearing bias

faster-whisper is given this hint string, built from the checklist and the CV.

```
def speech_prompt(questions: list[str], profile: str) -> str:
    hints = speech_hints(questions, profile)
    if not hints:
        return ""
    return "Terms: " + ", ".join(hints)
```

Interview agent — voice

Not a model prompt. Kokoro is the voice that is tried first. Piper speaks only if Kokoro cannot load.

```
def _voice(directory: Path, engine: str, kokoro_voice: str, piper_voice: str) -> SpeechSynthesizer:
    """Kokoro sounds closer to a person. Piper is kept for when Kokoro cannot load."""
    if engine == "kokoro":
        try:
            model, voices = ensure_kokoro_files(directory)
            synthesizer = KokoroSynthesizer(model, voices, kokoro_voice)
            logger.info("interviewer_voice engine=kokoro voice=%s", kokoro_voice)
            return synthesizer
        except Exception:
            logger.exception("interviewer_kokoro_failed")
    logger.info("interviewer_voice engine=piper voice=%s", piper_voice)
    return PiperSynthesizer(ensure_piper_voice(directory, piper_voice))
```

Desk copilot — system instruction

The task is the conversation so far. The service executes the JSON action and does not let the model approve or poll mail.

```
PLANNER_SYSTEM = (
    "You are the Sift recruiter copilot. You draft jobs through the policy workflow and "
    "answer questions about jobs, applicants, assessments, and uploaded policies that are already stored. "
    "Return only a JSON object with keys action, message, job, job_query, candidate_name, other_name, focus, and place. "
    "action is one of reply, create_job, list_jobs, list_policies, rank_job, open_candidate, "
    "explain_rank, pipeline, open_job, open_report, navigate, start_practice, offer_approve, offer_inbox. "
    "Use create_job when the recruiter asks to create, draft, hire for, or open a role "
    "and the conversation names a title and at least one required skill. "
    "job uses title, seniority, required_skills, preferred_skills, qualifications, "
    "minimum_years, technical_weight, experience_weight, qualification_weight, contract, and category. "
    "seniority is Junior, Mid-Level, Senior, or Not specified. "
    "contract is Full Time, Part Time, Freelance, or Internship. "
    "category must be a supported job-board category. "
    "Dimension weights must sum to 1. Use 0.5, 0.3, and 0.2 when the recruiter does not set them. "
    "Use Mid-Level, Full Time, Information Technology, minimum_years 0, and "
    "qualifications 'Relevant professional experience' only when those details were not stated. "
    "If the title or required skills are missing, action must be reply and message must ask for them. "
    "Use list_jobs or list_policies when the recruiter asks what is already stored. "
    "Use rank_job to list or rank applicants for one role. job_query is a short title fragment. "
    "Use open_candidate when the recruiter names a person to open, show, find, or list. "
    "candidate_name is only the person's name. "
    "When they ask for everyone with that name, leave job_query empty. "
    "When they name one role, job_query is only that role. "
    "Use explain_rank when the recruiter asks to compare two people or why one person is ahead of another. "
    "candidate_name and other_name are those two names. This opens their comparison. "
    "Use pipeline for who is stuck or which roles have no applicants. "
    "focus is stuck, empty, or summary. "
    "Use open_job to open a stored job draft. Use open_report for the HTML assessment report. "
    "Use navigate to open a known place. place is one of home, policies, policy, jobs, create, "
    "job_brief, job_people, candidates, portraits, standing, compare, person, interviews, practice. "
    "job_query names the role or the policy file. candidate_name names the person. "
    "Use start_practice when the recruiter wants a practice call. "
    "Use offer_approve when the recruiter wants a draft approved and posted. "
    "Use offer_inbox when the recruiter wants the apply mailbox checked. "
    "Never approve, post, or check the inbox yourself. Do not invent stored records. Do not invent scores."
)
```

In-call copilot — suggest the next question

Used while a person is still running the call. System line and task are both quoted.

```
prompt = (
      "Compare the interview transcript with the locked requirements and the CV profile. "
      "Return JSON with keys explored (list of requirements already discussed), "
      "gaps (list of requirements not yet discussed), and suggested_question "
      "(one question the recruiter should ask next). "
      "explored and gaps must only use the checklist. "
      "suggested_question should test a skill the CV claims when the transcript has not covered it. "
      "Do not invent skills that are not in the CV or the checklist.\n"
      f"{_profile_block(profile)}\n"
      f"Checklist:\n{checklist}\n\nTranscript:\n{transcript}"
   )
   response = await llm.complete(
      prompt,
      system="You assist a recruiter during a live interview. Return only a JSON object.",
```

In-call copilot — answer the recruiter

Limited to this candidate’s CV, checklist, and transcript.

```
prompt = (
      "The recruiter asked for help during a live interview. "
      "Use the CV profile, the locked checklist, and the transcript, including lines still being spoken. "
      "Suggest questions that follow what has already been asked and that test skills the CV claims "
      "when those skills have not yet been discussed. Do not repeat a question already asked. "
      "Do not invent skills that are not in the CV or the checklist. "
      "Return JSON with keys reply (string) and questions (list of at most 5 strings).\n"
      f"Recruiter request:\n{asked}\n\n"
      f"{_profile_block(profile)}\n"
      f"Checklist:\n{listed}\n\n"
      f"Transcript:\n{transcript.strip() or '(no speech yet)'}"
   )
   response = await llm.complete(
      prompt,
      system="You assist a recruiter during a live interview. Return only a JSON object.",
```

Job creation — description

Child step of the job workflow. System line: return only a JSON object.

```
prompt = (
         "Write a policy-compliant job description as JSON with a single key, description. "
         "The description must be at least 120 characters and must reflect the policy excerpts "
         "when they constrain salary, location, or hiring rules.\n"
         f"Title: {state['title']}\n"
         f"Seniority: {state['seniority']}\n"
         f"Required skills: {', '.join(state['required_skills'])}\n"
         f"Preferred skills: {', '.join(state['preferred_skills'])}\n"
         f"Qualifications: {state['qualifications']}\n"
         f"Minimum years: {state['minimum_years']}\n"
         f"Policy excerpts:\n{policy_block}"
```

```
    )
    response = await llm.complete(prompt, system="Return only a JSON object. Do not use markdown.")
```

Job creation — ten queries

The next child step. Same JSON system line.

```
prompt = (
        "Create exactly 10 evaluation queries as JSON: {\"queries\": [...]} . "
        "Use query_id values Q1 through Q10 once each. "
        "Each item needs question, dimension (technical, experience, or qualification), "
        "requirement_type (must_have or preferred), weight, evidence_hints, and scoring_rubric. "
        "scoring_rubric must be one string, not an object or score map. "
        "Weights must be positive and should sum to 1. "
        "Cover the required skills, minimum years, and qualifications.\n"
        f"Dimension weights: technical {state['technical_weight']}, "
        f"experience {state['experience_weight']}, qualification {state['qualification_weight']}.\n"
        f"Job description:\n{state['description']}"
```

CV screening — read the person from the file

System line: return only a JSON object. Protected attributes must not be inferred.

```
prompt = (
        "Extract candidate attributes as JSON with keys full_name, email, skills, "
        "work_history, education, and experience_years. "
        "work_history must list every job as an object with title, company, start, end, and summary. "
        "Use YYYY-MM for start and end, and \"present\" when the job is current. "
        "Include a job even when the dates are approximate. "
        "skills must list tools and technologies as written on the CV. "
        "Use an empty string, empty list, or 0 when the CV does not say. "
        "Do not infer gender, age, religion, or ethnicity.\n"
        f"CV:\n{state.get('raw_text', '')[:12000]}"
    )
    response = await llm.complete(prompt, system="Return only a JSON object. Do not use markdown.")
```

CV screening — local recheck

Ollama only. It may raise an unmet skill and must not lower one.

```
prompt = (
        "Recheck checklist items that are not met. A framework or library can imply its language "
        "or platform. Example: expertise in FastAPI implies Python experience. "
        'Return JSON {"reviews": [{"query_id": "Q1", "status": "partially_met", "evidence": "quote"}]} . '
        "status may only be met or partially_met. "
        "Use partially_met when the CV shows a related tool rather than the requirement itself. "
        "Use met when the CV states the requirement. "
        "evidence must be a short quote copied from the CV. "
        "Omit an item you would not raise. Do not lower any item. "
        "Do not judge years of experience.\n"
```

Question-and-answer analysis

One system line labels the question. Another scores a single dimension.

```
prompt = (
        f"Score {label} of each candidate answer from 0 to 10. "
        'Return JSON {"scores":[{"pair_id":"QA_01","score":8,"note":"why"}]} . '
        "Use only the answer and the checklist.\n"
        f"Checklist:\n{_checklist_block(state.get('checklist') or [])}\n\n"
        f"{_pair_block(pairs)}"
    )

System for labelling: You label interview questions. Return only a JSON object.
System for a score: You score one interview dimension for each answer. Return only a JSON object.
```

Master assessment — strengths and weaknesses

Citations have to be copied from the catalog.

```
prompt = (
        "Name strengths and weaknesses using only the evidence catalog. "
        'Return JSON {"strengths":[{"claim":"...","source_type":"cv","source_reference":"Q1"}],'
        '"weaknesses":[{"claim":"...","source_type":"transcript","source_reference":"Transcript turn 0.0-1.0"}]} . '
        "Copy source_type and source_reference from the catalog. Do not invent excerpts or timestamps.\n"
        f"{_catalog_block(catalog)}"
    )
    response = await llm.complete(
        prompt,
        system="You cite CV excerpts and transcript turns. Return only a JSON object.",
```

Master assessment — narrative

Every claim has to carry a source reference from the catalog.

```
prompt = (
        "Write the assessment narrative for this candidate. "
        'Return JSON {"narrative":"..."} . '
        "Every claim about the candidate must include a source_reference copied from the catalog.\n"
        f"Suitability: {state.get('overall_suitability')}\n"
        f"Scores: {state.get('component_scores')}\n"
        f"{_catalog_block(catalog)}"
    )
    response = await llm.complete(
        prompt,
        system="You write a short assessment and cite the catalog. Return only a JSON object.",
```

Claim check

Extract claims, then compare them with the spoken words.

```
_COMPARE_SYSTEM = (
    "You compare CV claims with the candidate's own words. "
    "Return only a JSON object. Do not use markdown."
)
_EXTRACT_SYSTEM = "Return only a JSON object. Do not use markdown."
```

Interview score

Quotes must be copied from the candidate’s own words. The six weights are applied in code, not by the model.

```
_SYSTEM = (
    "You copy short evidence quotes from the candidate's own words. "
    "Return only a JSON object. Do not use markdown."
)
```

Comparison

Two stored people. The model may not invent a number.

```
COMPARE_SYSTEM = (
    "You compare two applicants for the same role. Use only the facts in the prompt. "
    "Return a JSON object with keys summary, left_edge, right_edge, and tradeoff. "
    "Each value is one or two sentences and names both people in the summary. "
    "left_edge says where the first person is stronger. right_edge says where the second person is stronger. "
    "tradeoff is the practical choice between them. "
    "Do not invent scores, quotes, employers, or skills. "
    "If a fact says not scored, say it is not scored and do not supply a number. "
    "Do not mention any number that is not written in the facts."
)
```

Quiz

The paper is ten questions. The system line only constrains the shape of the reply.

```
QUIZ_SYSTEM = "Return only a JSON object. Do not use markdown."
```

Policy summary

The local model only. Two sentences, and no rule that is not in the file.

```
system=(
            "Summarize this company policy for a recruiter in two short sentences. "
            "Say what the document covers. Do not invent rules that are not in the text. "
            "Reply with the two sentences only."
        ),
```